Software Engineering Assessment — Java

Format: CodeSignal-style. Two parts, taken back to back.

Part	Content	Time	Weight
A — General Coding (GCA)	4 independent tasks, increasing difficulty	70 min	50 pts
B — Industry Coding (ICF)	1 problem, 4 progressive levels	90 min	50 pts

Level. Calibrated for a final-year engineering student (EMSI and similar) sitting a screening round at a consulting-tech employer. Part A matches the format candidates report from BCG X Casablanca: CodeSignal, four problems, seventy minutes. Nothing here needs a competitive-programming background — no dynamic programming, no shortest paths, no advanced data structures. What it does need is fluent Java collections, careful reading, and edge cases handled without being reminded.

Rules

- Edit only the files under `src/` . Do not touch `tests/` or `solutions/` .
- Java 17+ only. No external libraries, no internet.
- Part A: tasks are independent. If you are stuck, move on — a partially complete assessment beats a timed-out one.
- Part B: levels build on each other. Do **not** start a level before the previous one passes. Expect to refactor between levels; that is the skill being measured.

Run `./run.ps1` (or `./run.sh`) to grade yourself, or `./exam.ps1` for the timed environment.

Part A — General Coding Assessment

Time budget: 70 minutes total. Suggested split: 8 / 15 / 22 / 25.

Task 1 — Run-Length Compression (10 pts)

File: `src/gca/Task1.java`

```
public static String compress(String s)
```

Compress `s` by replacing every maximal run of identical characters with the character followed by the run length. A run of length 1 is written as just the character (no `1`).

Examples

Input	Output
`"aaabccddd"`	`"a3bc2d3"`
`"abc"`	`"abc"`
`"zzzzzzzzzzz"`	`"z12"`
`"aabbaa"`	`"a2b2a2"`
`""`	`""`

Constraints

- `0 <= s.length() <= 100000`
- `s` contains lowercase English letters only.
- `s` may be `null` — return `""`.

Target: O(n) time. Build the result with a `StringBuilder`, not `String +=`.

Task 2 — Department Averages (12 pts)

File: `src/gca/Task2.java`

```
public static String[] topDepartments(String[] records, int n)
```

Each record is a line of the form `"name,department,salary"` . Group the records by department, compute each department's average salary, and return the top `n` departments formatted as `"department(average)"` .

- The average is an **integer division** — it rounds down. `21 / 2` is `10` .
- Order by average **descending**, then by department name **ascending**.
- Fields are trimmed: `"a, eng , 100 "` is the department `eng` , salary `100` .
- A record is **ignored** if it does not have exactly three comma-separated fields, if the department is empty, or if the salary is not a whole number `>= 0` .
- Return an empty array (not `null`) if nothing qualifies or `n <= 0` .

Examples

```
String[] records = {
    "ana,eng,100", "bo,eng,200", "cy,sales,150", "dee,sales,150", "bad record"
};

topDepartments(records, 2)   // ["eng(150)", "sales(150)"]   both average 150, name breaks the tie
topDepartments(records, 1)   // ["eng(150)"]
topDepartments(records, 99)  // ["eng(150)", "sales(150)"]

topDepartments(new String[] {"a,x,10", "b,x,11"}, 5)  // ["x(10)"]   21/2 rounds down
topDepartments(null, 5)                               // []
```

Constraints

- `0 <= records.length <= 200000` ; `records` and its entries may be `null` .
- `0 <= salary <= 2000000000` — a department's **total** can exceed `int` .
- Target: O(R + D log D) for R records and D departments.

Task 3 — Minesweeper Reveal (14 pts)

File: `src/gca/Task3.java`

```
public static char[][] reveal(char[][] board, int r, int c)
```

`board` is a minesweeper grid of hidden cells:

- `'.'` — hidden empty cell
- `'M'` — hidden mine

The player clicks cell `(r, c)`. Mutate `board` in place and return it:

1. If the clicked cell is a mine (`'M'`), set it to `'X'` and stop. Nothing else changes.
2. Otherwise, reveal the clicked cell:
   - If it has at least one mine among its **8** neighbours, set it to the digit character for that count (`'1'` .. `'8'`) and stop expanding from it.
   - If it has zero adjacent mines, set it to `'B'` (blank) and recursively reveal all 8 of its still-hidden empty neighbours by the same rules.
3. If the clicked cell is already revealed (a digit, `'B'`, or `'X'`), or the coordinates are out of bounds, return the board unchanged.

Mines are never revealed by the cascade — only by a direct click.

Example

```
input board           click (3, 0)        result
. . . . .                                 B 1 . 1 B
. . M . .             ----------->        B 1 M 1 B
. . . . .                                 B 1 1 1 B
. . . . .                                 B B B B B
```

The cascade stops at the eight cells touching the mine, so those become `1`. Cell `(0,2)` stays hidden: every one of its neighbours is either the mine or a numbered cell, so the flood never reaches it.

Constraints

- `1 <= board.length, board[0].length <= 300`
- `board` is rectangular and non-null; `r`, `c` may be out of range.

Target: O(rows · cols). Use an explicit stack or queue — a recursive flood fill can blow the stack on a 300×300 empty board.

Task 4 — Longest Balanced Segment (14 pts)

File: `src/gca/Task4.java`

```
public static int longestBalancedSegment(int[] nums)
```

Return the length of the longest **contiguous** subarray of `nums` that contains exactly as many even numbers as odd numbers. Return `0` if no such subarray exists.

Examples

Input	Output	Why
`[1,2,3,4,5]`	`4`	`[1,2,3,4]` — 2 odd, 2 even
`[2,4,6]`	`0`	no odd numbers at all
`[1,2]`	`2`	
`[1,1,2,2,1]`	`4`	`[1,1,2,2]`
`[-1,-2,3,4]`	`4`	negatives count normally (`-2` is even)
`[]`	`0`	

Constraints

- `0 <= nums.length <= 200000`
- `-10^9 <= nums[i] <= 10^9`
- `nums` may be `null` — return `0` .

Target: O(n) time. An O(n²) solution will time out on the large case.

> Hint: a subarray is balanced iff a running counter (+1 per odd, –1 per even) has the same value at both ends. For each value of that counter, only the earliest index it appeared at matters.

Part B — Industry Coding Framework

File: `src/icf/CloudStorage.java` **Time budget:** 90 minutes. Suggested split: 10 / 20 / 30 / 30.

You are implementing the backend of an in-memory cloud storage service. All state lives in one `CloudStorage` instance; there is no persistence and no concurrency.

Each level adds requirements. **Signatures are fixed** — the grader calls them exactly as written. Everything specified at an earlier level must keep working.

File names are opaque strings that look like paths (`"/dir/file.txt"`), but you never need to parse them as paths — prefix matching is plain string prefix matching.

Level 1 — Files (10 pts)

```
public boolean addFile(String name, int size)
```

Add a new file. Returns `true` on success, `false` if a file with that name already exists.

```
public Integer getFileSize(String name)
```

Returns the file's size, or `null` if there is no such file.

```
public Integer deleteFile(String name)
```

Deletes the file and returns its size, or `null` if there is no such file.

Example

```
addFile("/dir/file.txt", 10)   // true
addFile("/dir/file.txt", 20)   // false
getFileSize("/dir/file.txt")   // 10
deleteFile("/dir/file.txt")    // 10
deleteFile("/dir/file.txt")    // null
getFileSize("/dir/file.txt")   // null
```

Level 2 — Queries (10 pts)

```
public String[] getNLargest(String prefix, int n)
```

Returns at most `n` files whose name starts with `prefix`, as strings of the form `"name(size)"`.

Ordering: by size **descending**; ties broken by name **ascending** (lexicographic, `String.compareTo`).

Returns an empty array (not `null`) if nothing matches or `n <= 0`.

Example

```
addFile("/dir/f1", 10);
addFile("/dir/f2", 20);
addFile("/dir/deeper/f3", 30);
addFile("/other/f4", 40);

getNLargest("/dir", 2)   // ["/dir/deeper/f3(30)", "/dir/f2(20)"]
getNLargest("/dir/", 10) // ["/dir/deeper/f3(30)", "/dir/f2(20)", "/dir/f1(10)"]
getNLargest("/nope", 5)  // []
```

Level 3 — Users and Capacity (15 pts)

Files now have an owner. A user has a fixed storage **capacity**; the sum of their files' sizes may never exceed it.

```
public boolean addUser(String userId, int capacity)
```

Creates a user. Returns `false` if `userId` already exists.

```
public Integer addFileBy(String userId, String name, int size)
```

Adds a file owned by `userId` and returns that user's **remaining** capacity after the add. Returns `null` and changes nothing if the user does not exist, the file name is taken, or the add would exceed the user's capacity.

There is a built-in user `"admin"` with unlimited capacity, created with the `CloudStorage` instance. `addFile(name, size)` from Level 1 is equivalent to `addFileBy("admin", name, size)`. For `"admin"`, remaining capacity is reported as `Integer.MAX_VALUE`.

`deleteFile` must free the owner's capacity.

```
public Integer mergeUser(String userId1, String userId2)
```

Transfers every file owned by `userId2` to `userId1`, adds `userId2`'s capacity to `userId1`'s, then deletes `userId2`. Returns `userId1`'s remaining capacity.

Returns `null` and changes nothing if either user does not exist, if `userId1.equals(userId2)`, or if `userId2` is `"admin"`.

Example

```
addUser("u1", 100)            // true
addUser("u1", 50)             // false
addFileBy("u1", "/u1/f", 50)  // 50
addFileBy("u1", "/u1/g", 60)  // null  (would exceed capacity)
addFileBy("ghost", "/x", 1)   // null  (no such user)

addUser("u2", 50)             // true
addFileBy("u2", "/u2/h", 20)  // 30
mergeUser("u1", "u2")         // 80
mergeUser("u1", "u2")         // null  (u2 no longer exists)
getFileSize("/u2/h")          // 20    (file survived, now owned by u1)
```

> The merge result: `u1` had capacity 100 with 50 used, `u2` had capacity 50 with 20 used. Afterwards `u1` has capacity 150 and 70 used — remaining `80`.

Level 4 — Backup and Restore (15 pts)

```
public Integer backupUser(String userId)
```

Snapshots the names and sizes of all files currently owned by `userId` , replacing any previous snapshot for that user. Returns the number of files backed up, or `null` if the user does not exist.

```
public Integer restoreUser(String userId)
```

Deletes all files currently owned by `userId` , then re-adds the files from that user's snapshot. A snapshot file whose name has since been taken by **another** user is skipped. Returns the number of files actually restored, or `null` if the user does not exist.

If the user has no snapshot, treat the snapshot as empty: delete all of their files and return `0` .

Capacity accounting must end up consistent — a user's remaining capacity after a restore is their capacity minus the total size of the restored files.

A user deleted by `mergeUser` loses its snapshot.

Example

```
addUser("u1", 100);
addFileBy("u1", "/a", 10);
addFileBy("u1", "/b", 20);
backupUser("u1")              // 2
addFileBy("u1", "/c", 30);
deleteFile("/a");
restoreUser("u1")             // 2
getFileSize("/a")             // 10
getFileSize("/c")             // null
backupUser("ghost")           // null
```

Scoring

```
Task 1  10      Level 1  10
Task 2  12      Level 2  10
Task 3  14      Level 3  15
Task 4  14      Level 4  15
-------------   -------------
Part A  50      Part B   50      Total 100
```

Within a task, points are awarded proportionally to tests passed. 117 checks in total.